

From Dense Port Tables to Polynomial Sparse Incidence

Certified orbit-query acceleration for compressed unknot hierarchies

Research contribution prepared for Vladimir Reshetnikov’s ProveIt project

8 October 2026

Abstract

The maintained marked-component observer in ProveIt’s unknot-recognition program uses exact interval-pairing orbit counts, canonical marked-union reuse, and a dense Boolean Möbius transform. Its output has 2^r entries for r ports even when only a few component signatures occur. We replace that dense representation by an exact positive sparse-zeta reconstruction, using the existing unweighted Agol–Hass–Thurston (AHT) kernel without expanding a single interval point.

For s nonempty signatures, recovery requires $O(sr)$ oracle queries. Balanced block deletion sharpens this to $O(1 + s + \sum_T |T| \log(1 + r/|T|))$. An independent certificate checks each positive signature through its immediate proper subsets and a final mass identity, rather than re-running discovery or enumerating all subsets. Crucially, for interval-union ports, polynomial support size already follows from the classical weighted AHT theorem. Consequently the sparse observer and its certificates have polynomial bit complexity in the encoded input, with no logarithmic restriction on the number of ports. This is a certified unweighted-query realization of a consequence of existing weighted orbit theory, not a new general weighted-orbit complexity theorem.

We also recover incidence classified by a supplied binary orientation character; the lifted histogram has exactly the same support, so its reconstruction needs no second support search. A standard-library implementation, two byte-identical pinned upstream kernels, independent literal-graph audits, adversarial tests, and paired measurements accompany the proofs. Several sparse families improve substantially; dense and small controls exhibit overhead. The result removes a specific exponential-format bottleneck. It neither constructs a complete unknot hierarchy nor proves a general quasi-polynomial unknot recognizer.

Status. Complete written proofs of the reduction, extraction, certificate, and binary-cover statements; a polynomial-complexity corollary explicitly dependent on classical AHT. Executable exact-arithmetic validation; no Lean formalization or first-in-literature priority claim. The delivered code is an additive research package, not a committed production change.

Contents

1	Research objective and the inspected bottleneck	4
1.1	Why this is the next component to improve	4
1.2	Snapshot and reproducibility boundary	4
1.3	The strongest result, and its attribution	5
2	Exact input and output models	5
2.1	Three sizes that must not be conflated	6
3	From marked unions to positive zeta queries	6
3.1	Coning without expanding a marked interval	6
3.2	The Boolean zeta function	7
3.3	Caching keys that preserve the exact question	7
4	Positive sparse reconstruction	7
4.1	Residuals and minimal positive supports	7
4.2	Flat deletion algorithm	8
4.3	Balanced block deletion	8
4.4	A useful information lower bound	9
5	Sparse certificates without dense forward sums	10
5.1	The algebraic certificate criterion	10
5.2	Why deletion witnesses cannot simply be discarded	10
5.3	Binding algebraic values to supplied geometry	11
5.4	What the local orbit proofs establish	11
6	Bit complexity and polynomial support	11
6.1	Output-sensitive accounting	11
6.2	Why the sparse support is polynomial for interval inputs	12
6.3	Dense output remains an exponential requirement	13
7	Incidence refined by a binary orientation character	14
7.1	Order reversal and orientation parity are different inputs	14
8	Counterexamples to overstrong conclusions	15
8.1	Signed weights invalidate zero-based extraction	15
8.2	Equal incidence is not equal future attachment behavior	15
8.3	Polynomial local work does not bound hierarchy search	15
9	A precise interface to a future quasi-polynomial hierarchy	16

10 Implementation contracts and resource behavior	16
10.1 Module separation	16
10.2 Completion, interruption, and mutable state	17
10.3 Serialization and hostile-input scope	17
11 Exact validation and controlled measurements	17
11.1 Completed validation	17
11.2 Benchmark design	18
11.3 Interpretation, including regressions	19
11.4 Large-port scaling without a dense allocation	19
12 Integration plan and preservation of existing guarantees	20
13 Further research questions and concrete next targets	20
13.1 A weighted engine versus repeated unweighted queries	20
13.2 Sharper support bounds tied to geometry	21
13.3 Residual-query data structures	21
13.4 Cost-aware balanced trees and port equivalences	21
13.5 Certificate size and streaming replay	21
13.6 Finite covers beyond a binary character	21
13.7 Attachment-compatible refinements	21
13.8 Certified port extraction and compressed cutting	22
13.9 Global potential and search accounting	22
13.10 Native validation and formal proof boundaries	22
14 Conclusion	22
A Executable usage and artifact inventory	23
B Worked sparse certificate on a mixed relation	24
C Claim ledger	24

1 Research objective and the inspected bottleneck

1.1 Why this is the next component to improve

The requested objective is to continue improving the exact recognizer in `Topology/UnknotRecognition`, aiming ultimately at quasi-polynomial asymptotic complexity. The inspected sources distinguish a complete recognition pipeline from local filters, compressed group operations, normal-surface observers, and hierarchy primitives. That distinction is essential: improving one exact primitive is useful, but does not establish a bound for a composition whose other stages or number of calls remain uncontrolled.

The most recent maintained orbit work is a particularly suitable target. It supplies an unweighted AHT count kernel, local proof replay, and a marked incidence observer. The synthesis explains that repeated unions are cached, but that the histogram and its proof-index table still have 2^r slots [4, 5]. Thus the representation itself can be exponentially larger than its nonzero information. This paper changes the representation, the discovery algorithm, and the certificate contract together.

The concrete outcome is a function taking a supplied compressed interval relation and marked interval unions and returning the *complete sparse histogram* of the component signatures. The result includes multiplicity, including the multiplicity of the unmarked signature. It is not merely a sample of components or a list of signatures encountered before a resource limit.

1.2 Snapshot and reproducibility boundary

The inspected repository ref is

7518823550fbc8c217bc8be0113fe52002e77465.

Two modules are delivered unchanged, with their Git blob identities checked before they are loaded:

Module	Git blob
<code>interval_orbits.py</code>	<code>e86050aadbcef6f17fdddf9cbd1abc0e7ca18e8</code>
<code>interval_orbit_verify.py</code>	<code>0ccb56a0e8b5f1255384121d7417441314727720</code>

The second module imports neither the orbit producer nor its scheduling helpers. Its proof checker is reused as a trusted local-relation layer; the new sparse certificate verifier does not call the sparse producer either.

All delivered measurements use the pinned count module with its `periodic_rule='aht'` setting, retaining the classical sufficient merger criterion. The sharper Fine–Wilf implementation remains unchanged in the vendored file, but its acceleration is not needed for this paper’s complexity argument or benchmark comparison. This choice avoids making the new argument depend on a separate analysis of a modified orbit scheduler.

The dense comparison arm is a small research ablation: identical coning, canonical-union caching, the same pinned classical kernel, and dense Möbius inversion. It is *not* a byte-for-byte copy of the full maintained incidence module or recognition dispatch. No full upstream test suite, Regina integration, native normal-surface corpus, or whole-knot benchmark is claimed to have been run for this contribution.

1.3 The strongest result, and its attribution

The important point is stronger than a conditional statement for small s . AHT's weighted algorithm returns compressed blocks of equal orbit-weight vectors in polynomial time [1, Section 6, Theorem 16]. Marking each port with one nonnegative coordinate shows that the number of distinct signatures is already polynomial in the interval-encoded input. The new unweighted-query reconstruction inherits that polynomial bound.

Accordingly, the contribution is not that weighted orbit information has suddenly become polynomial-time computable. It is an explicit, independently certifiable route from the already maintained *unweighted* engine to a sparse incidence interface, avoiding a second weighted-transfer implementation and avoiding the maintained dense output format. The output-sensitive oracle bound, the sparse local certificate, the support-reusing binary-cover extension, and the measured implementation are the repository-facing developments.

2 Exact input and output models

Definition 2.1 (Interval relation). *Let $X_N = \{0, \dots, N-1\}$, where $N \geq 0$ is binary encoded. An interval pairing is an isometry between two nonempty integer intervals of equal length. In the supplied representation it is a row (a, b, c, d, ϵ) with inclusive endpoints, $b - a = d - c$, and $\epsilon \in \{1, -1\}$. Its action is*

$$x \mapsto \begin{cases} x + c - a, & \epsilon = 1, \\ a + d - x, & \epsilon = -1. \end{cases}$$

The relation $\sim$ is the equivalence relation generated by k such pairings. Its equivalence classes are called orbits or components.

Pairings and their inverses generate the same equivalence relation. The input normalizer places the domain interval no farther right than the range, retaining the order of the supplied pairing list. Identity pairings and repeated pairings are allowed. The empty universe requires an empty pairing list. No input contains one record per point unless the caller chooses such an expanded representation.

Definition 2.2 (Ports and signatures). *A port $A_i \subseteq X_N$, $1 \leq i \leq r$, is supplied as a finite union of half-open intervals $[u, v)$. Endpoints satisfy $0 \leq u \leq v \leq N$. Within each port, overlap and adjacency are merged and empty intervals are removed. Ports may overlap, coincide, or be empty. For a component O , put*

$$\sigma(O) = \{i \in [r] : O \cap A_i \neq \emptyset\}, \quad h(T) = \#\{O : \sigma(O) = T\}.$$

The total component count is $C = \sum_{T \subseteq [r]} h(T)$.

Let M be the number of marked intervals in the input, or any upper bound on the number after normalization. Let

$$B = \max\{1, \lceil \log_2(N+1) \rceil\}, \quad s = \#\{T \neq \emptyset : h(T) > 0\}, \quad s_0 = \#\{T : h(T) > 0\}.$$

Thus s_0 is either s or $s+1$. The output is a sorted list of the s_0 positive pairs $(T, h(T))$, with a support encoded as an r -bit mask. An absent entry means exactly zero. The empty output means $C = 0$; it does not mean an unfinished computation.

Counts satisfy $0 \leq h(T) \leq C \leq N$, so every count uses $O(B)$ bits. The output needs $O(s_0(r + B))$ bits, apart from ordinary serialization conventions. A dense array instead has 2^r slots even for one positive signature. These are different output contracts, not interchangeable representations with an identical mandatory cost.

2.1 Three sizes that must not be conflated

The number of points N , the number of components C , and the number of signature types s_0 can differ by enormous factors. For a static universe with no pairings, $C = N$. If every point has the same nonempty signature, then $s_0 = 1$. Both N and C can equal 2^{16000} while the multiplicity is represented by one binary integer. A valid sparse algorithm must remove a whole signature's multiplicity in one step; an algorithm that removes one component at a time would lose the intended compressed complexity.

3 From marked unions to positive zeta queries

3.1 Coning without expanding a marked interval

For a selected union $A \subseteq X_N$, define

$$H(A) = \#\{O : O \cap A \neq \emptyset\}.$$

Suppose the canonical union is nonempty and consists of m_A disjoint, nonadjacent half-open intervals. Choose its first point as an anchor. For an interval $[u, v)$ of length at least two, add the pairing

$$[u, v - 2] \longrightarrow [u + 1, v - 1].$$

This is one inclusive interval pairing, not $v - u - 1$ separately stored edges. For every interval other than the first, add one singleton pairing between its first point and the anchor. At most $2m_A - 1 \leq 2M$ pairings are added. For a length-one first interval there is no internal translation.

Lemma 3.1 (Coning identity). *Let C_A be the component count after the preceding relations are added to the original interval system. If $A \neq \emptyset$, then*

$$H(A) = C - C_A + 1.$$

For the empty marked union, $H(\emptyset) = 0$ without an additional query.

Proof. The internal translation identifies consecutive points in each marked interval. The singleton anchor relations then identify all marked points. Every original component meeting A therefore joins the same new component. No added relation touches an original component disjoint from A , so those components remain distinct and unchanged. Exactly $H(A)$ old components become one, giving $C_A = C - H(A) + 1$. $\square$

The argument applies to overlapping ports, disconnected original systems, static points, and orientation-reversing original pairings. Coning is used only to ask a question about the original relation; the resulting merged system is not substituted for that relation in later geometry.

3.2 The Boolean zeta function

For $S, U \subseteq [r]$, put

$$F(S) = H\left(\bigcup_{i \in S} A_i\right), \quad G(U) = C - F([r] \setminus U).$$

Lemma 3.2 (Positive downset representation).

$$G(U) = \sum_{T \subseteq U} h(T).$$

In particular, $G([r]) = C$, $G(\emptyset) = h(\emptyset)$, and G is monotone under inclusion.

Proof. A component is excluded from $F([r] \setminus U)$ precisely when it meets none of the ports outside U . This is equivalent to its signature being a subset of U . Counting those components proves the formula. Monotonicity follows because every summand is nonnegative. $\square$

For a nonempty actual union of the excluded ports, Lemma 3.1 gives the useful direct formula

$$G(U) = C_A - 1. \tag{1}$$

When that actual union is empty, $G(U) = C$. A nonempty mask can describe an empty union if all its selected ports are empty; testing the mask alone is therefore insufficient.

3.3 Caching keys that preserve the exact question

The oracle stores the baseline count once. It also stores completed $G(U)$ values by mask and coned counts by the *canonical actual union* of the excluded ports. Distinct masks with the same union produce exactly the same coning rows in the same order, so a count and its proof can safely be reused. Equality of two counts is never used as evidence that two unions are interchangeable.

This retains the maintained observer's useful sharing mechanism. What changes is that only the masks needed by discovery or certification are constructed. A cache indexed by all 2^r masks in advance would reintroduce the bottleneck.

4 Positive sparse reconstruction

4.1 Residuals and minimal positive supports

Let $\mathcal{K}$ be a set of signatures whose multiplicities have already been recovered exactly. Define

$$R_{\mathcal{K}}(U) = G(U) - \sum_{T \in \mathcal{K}, T \subseteq U} h(T). \tag{2}$$

Under the stated invariant,

$$R_{\mathcal{K}}(U) = \sum_{T \notin \mathcal{K}, T \subseteq U} h(T).$$

Thus the residual is again a nonnegative, monotone zeta function. In particular, a zero residual rules out every unrecovered signature in the queried downset; it is not merely a cancellation among positive and negative terms.

Lemma 4.1 (Minimal positive extraction). *If S is inclusion-minimal among sets with $R_{\mathcal{K}}(S) > 0$, then $S \notin \mathcal{K}$, $h(S) > 0$, and $R_{\mathcal{K}}(S) = h(S)$.*

Proof. Every unrecovered proper subset $T \subsetneq S$ lies inside some $S \setminus \{a\}$, $a \in S \setminus T$. Minimality says that the residual there is zero, so nonnegativity forces $h(T) = 0$. The only remaining possible positive summand at S is $h(S)$. Since the residual is positive, that summand exists and has not already been removed. $\square$

The multiplicity is therefore recovered exactly in one subtraction. It may be exponential in B as an ordinary integer magnitude, but its bit length is only $O(B)$.

4.2 Flat deletion algorithm

First obtain $C = G([r])$ and $h(\emptyset) = G(\emptyset)$. Retain the empty signature if its multiplicity is positive. While the sum of recovered multiplicities is smaller than C , start with $S = [r]$ and inspect the ports in a fixed order. Delete a from S whenever $R_{\mathcal{K}}(S \setminus \{a\}) > 0$. At the end, recover $h(S) = R_{\mathcal{K}}(S)$ and repeat.

Theorem 4.2 (Exact flat recovery). *For every nonnegative Boolean histogram, the preceding algorithm returns the complete sparse histogram after exactly s nonempty-support extraction iterations. It needs at most $2 + s(r + 1)$ logical zeta requests, before caching, and does not require advance knowledge of s or of the support sizes.*

Proof. At the start of each iteration, the residual at $[r]$ is positive because some total mass remains. Each successful deletion preserves that property. Suppose a deletion of a failed at an intermediate set S' . The final set S is contained in S' , so $S \setminus \{a\} \subseteq S' \setminus \{a\}$. Monotonicity keeps its residual zero. Hence every retained element fails a final single-element deletion, and every proper subset of S is contained in one such deletion. The final S is minimal positive. Lemma 4.1 therefore recovers one exact new nonempty signature.

The empty signature has already been removed, so this iteration cannot return it. An iteration subtracts the full positive mass of one previously unknown signature and never changes any other multiplicity. Exactly s iterations suffice. When the recovered mass is C , no missing positive entry can remain. The initial two requests, at most r deletion tests, and one final weight request per iteration give the stated bound. $\square$

The implementation computes the remaining total by an integer mass accumulator; it does not re-query the full set at each iteration. A mask cache can also eliminate repeated requests for the empty set or a final support.

4.3 Balanced block deletion

Deleting one port at a time wastes queries when an unknown signature is small. Partition the ordered r ports into a balanced binary interval tree. At a current set S , visit a tree block D and test $R_{\mathcal{K}}(S \setminus D)$. If positive, delete the entire block and do not visit its children. If zero and the block is not a leaf, recursively visit its two children in their fixed order. A failed leaf is retained. Start by visiting the root block; its empty-set query is normally already cached.

Theorem 4.3 (Support-sensitive query bound). *Balanced block deletion returns a minimal positive support. If that support has size $d \geq 1$, its number of block tests is at most*

$$b(r, d) = 1 + 2 \sum_{\ell=0}^{\lceil \log_2 r \rceil - 1} \min(2^\ell, d) = O\left(1 + d \log\left(1 + \frac{r}{d}\right)\right). \quad (3)$$

Consequently the entire histogram is recovered using

$$Q = O\left(1 + s + \sum_{T \neq \emptyset: h(T) > 0} |T| \log\left(1 + \frac{r}{|T|}\right)\right) \quad (4)$$

zeta requests, and in particular $Q = O(1 + sr)$. For $r = 0$, only the baseline/empty count is needed.

Proof. Every successful block deletion preserves positive residual. Every retained leaf has failed its deletion test, and the monotonicity argument from Theorem 4.2 keeps that failure valid as subsequent deletions shrink S . Thus the final set is minimal positive.

Fix this final support T . Whenever a block D fails at an intermediate set S' , all residual signatures contained in S' meet D . The final T is one such signature, so $D \cap T \neq \emptyset$. At depth ℓ , the tree blocks are disjoint. At most $\min(2^\ell, d)$ blocks at that depth can fail. Every visited block other than the root is a child of a failed nonleaf. There are therefore at most twice that many visits at the next depth. Summing gives (3); an unbalanced last level only reduces the count.

To estimate the sum, levels with $2^\ell \leq d$ contribute $O(d)$ in total, and the remaining $O(1 + \log(r/d))$ levels contribute $O(d)$ each. One additional request obtains the weight of the final support. Add the initial requests and sum over all extraction iterations to obtain (4). $\square$

Remark 4.4 (Order influences cost, not validity). *The final minimal signature and the number of requests can depend on the port order. Every fixed order is correct, and the displayed bound does not assume a favorable ordering. Finding an order minimizing expensive coned queries is a separate optimization problem. A query count alone does not measure the differing cost of its underlying interval systems.*

4.4 A useful information lower bound

Proposition 4.5 (Single-signature lower bound). *Suppose an oracle has one unknown positive signature T of known size d and known weight one, on r ports. Any deterministic adaptive reconstruction algorithm using zeta queries needs at least $\lceil \log_2 \binom{r}{d} \rceil$ queries in its worst case.*

Proof. Every query returns either zero or one, according as $T \not\subseteq U$ or $T \subseteq U$. A depth- q binary decision tree has at most 2^q leaves, whereas the $\binom{r}{d}$ possible supports must be distinguished. $\square$

For sparse ranks away from the degenerate dense endpoint, this gives the same familiar $d \log(r/d)$ information scale as balanced deletion. It is not an optimality claim for every d , for multiple weighted signatures, or for special interval geometry. For example, if $d = r$ is promised in advance, the support is already known; our general-purpose algorithm does not use that promise.

Hidden weighted-hypergraph reconstruction is an established query-learning subject. Bshouty and Mazzawi obtain substantially more refined nonadaptive bounds for constant-rank weighted hypergraphs [2]. Here the requirements are different: arbitrary support sizes, simple exact adaptive queries, compatibility with compressed interval geometry, and an independent certificate for the complete returned histogram. No general query-optimality or priority claim is made.

5 Sparse certificates without dense forward sums

5.1 The algebraic certificate criterion

Let proposed positive entries be $(T_1, w_1), \dots, (T_t, w_t)$, with distinct nonempty masks and positive integer weights. A canonical certificate orders them first by cardinality, then by mask. Let $w_0 \geq 0$ be the proposed empty multiplicity, and define

$$K_j(U) = w_0 + \sum_{i < j: T_i \subseteq U} w_i.$$

The w_0 term occurs in every downset, including the empty downset.

Theorem 5.1 (Local sparse certificate). *Assume G is the zeta transform of a nonnegative histogram of total mass C . The proposed entries are exactly that histogram if the following equalities hold:*

$$G(\emptyset) = w_0, \tag{5}$$

$$G(T_j) = K_j(T_j) + w_j \tag{6} \quad (1 \leq j \leq t),$$

$$G(T_j \setminus \{a\}) = K_j(T_j \setminus \{a\}) \tag{7} \quad (a \in T_j, 1 \leq j \leq t),$$

$$C = w_0 + \sum_{j=1}^t w_j. \tag{8}$$

Including the baseline, at most $2 + t + \sum_j |T_j|$ orbit-count queries are sufficient before sharing of coincident unions and empty-union shortcuts.

Proof. Equation (5) proves the empty multiplicity. Inductively assume the earlier nonempty entries have also been proved correct. Subtracting those entries leaves an actual nonnegative residual histogram. Equation (7) says that its zeta value vanishes on every immediate proper subset of T_j . Every proper subset is contained in one of these immediate subsets. Nonnegativity therefore forces every residual proper-subset weight to be zero. Equation (6) then proves that the actual residual weight at T_j is precisely w_j .

After all entries have been checked, every proposed multiplicity has been proved exact. The remaining mass is nonnegative, and (8) says that it is zero. Hence there are no omitted positive signatures anywhere, not just inside the tested downsets. The query bound counts one baseline, one empty-set value, and the support and immediate proper-subset values for every nonempty entry. $\square$

This certificate does not require the verifier to know which supports the producer tested, which block order it chose, or why it stopped searching. It verifies the answer, rather than the execution strategy. The final mass identity is indispensable: local correctness of several entries alone says nothing about signatures outside their downsets.

5.2 Why deletion witnesses cannot simply be discarded

Consider a proposed histogram with a single weight-one support $[r]$. Its total mass and full-set value agree with the histogram having support $[r] \setminus \{a\}$ for any fixed a , provided $r \geq 2$. Among all subsets $U \subseteq [r]$, the corresponding zeta functions differ only at $U = [r] \setminus \{a\}$. Thus a general exact verifier for this claimed full-support histogram must distinguish each such alternative. It needs all r immediate-deletion values in this oracle model. A final mass check plus one value per output row is not, in general, a sound replacement.

5.3 Binding algebraic values to supplied geometry

The delivered certificate has the schema `sparse-incidence-v1`. It contains the canonical input universe, ordered pairing rows, ordered normalized ports, a baseline local orbit proof, the sparse entries, and a list of coned union proofs. The producer first recovers the histogram, then obtains any additional values required by Theorem 5.1. Those additional calls use the *same remaining* query and cycle allowances. It emits only proofs needed by the certificate, not proofs of discovery-only queries.

The new verifier performs the following checks. It validates and compares the complete input binding, rejecting noncanonical certificate geometry and Boolean values masquerading as integer fields. It replays the baseline proof with the independent upstream local checker. For each requested zeta value, it reconstructs the actual excluded-port union with an independent endpoint sweep, rebuilds the coning rows, and replays the bound orbit proof. It then checks (5)–(8). Missing, duplicate, unused, or misbound union proofs are rejected. Repeated equal unions are checked once, but never equated merely because their claimed counts match.

The verifier imports no reconstruction algorithm or orbit producer. It shares only public geometry normalization with the producer. In the tests, the orbit producer is deliberately disabled after proof construction and the completed sparse proof still verifies.

5.4 What the local orbit proofs establish

The reused upstream verifier recognizes identity deletion, static contraction, reflection trimming, periodic merger, transmission by a certified power of a partial inverse, and exposed suffix truncation [4]. It checks those relations locally instead of insisting on the producer’s preferred scheduling choices. A proof must finish with an empty universe and account for the claimed number of removed singleton orbits. An unfinished valid prefix is not an orbit-count certificate.

The algebraic certificate theorem depends only on truthful query values, and the coning lemma binds those values to the original components. Thus the proof layers have separate obligations: local relation replay proves counts, coning proves marked-union interpretation, and nonnegative sparse algebra proves completeness of the histogram. None of these layers establishes that the supplied interval system is the correct knot-exterior model of a diagram.

6 Bit complexity and polynomial support

6.1 Output-sensitive accounting

Let $P(K, B)$ bound a completed classical unweighted AHT call, including optional recording when needed, on K pairings and B -bit endpoints. Let $V(K, B)$ and $L(K, B)$ bound independent replay time and serialized binary trace size for such produced proofs. These bounds are polynomial by classical AHT and the local trace construction; precise optimized exponents are not needed here. Logs in these bounds may be replaced by $\max(1, \log)$ to cover zero and unit-sized inputs.

Every coned query has $K \leq k + 2M$ pairings on the *same* universe. No coning operation increases endpoint bit length. A straightforward sorted union costs $O(M \log(M + 1)B)$ bit operations, with additional mask operations polynomial in r . The delivered residual calculation scans the list of previous positive entries. For each pair it performs an r -bit containment test and, when needed, a B -bit subtraction.

Theorem 6.1 (A conservative sparse bit bound). *With Q as in (4), histogram discovery can be implemented in bit time*

$$O(Q[P(k + 2M, B) + M \log(M + 1)B + (s + 1)(r + B)]) + \text{poly}(k + M + r + B). \quad (9)$$

The analogous certificate construction adds at most $Q_c = 2 + s + \sum_T |T|$ potential count queries and polynomial sparse bookkeeping. Verification replaces P by V for its at most Q_c distinct proofs. No 2^r term is required in discovery, certificate output, or replay.

Proof. Use Lemma 3.1 for every required marked-union value and charge each actual count by $P(k + 2M, B)$. Building the corresponding union and coning rows has the stated sorting and linear-output cost. For a residual value, at most $s + 1$ already recovered masks and counts are inspected. All residual values are between zero and C , so their integers have $O(B)$ bits. Add the input normalization, sparse output sorting, and initial allocations to the polynomial term. The certificate query bound is Theorem 5.1; its verifier uses only those masks and the supplied traces. $\square$

The displayed estimate deliberately does not assume constant-time arithmetic on arbitrarily large Python integers. Dictionary lookups in the implementation use ordinary expected hash-table costs. A deterministic ordered dictionary would add polynomial comparison factors, without changing the polynomial conclusion. A still simpler list-based cache also suffices for that conclusion.

Proof storage is bounded by

$$O(Q_c L(k + 2M, B) + Q_c MB + s_0(r + B)) + \text{poly}(k + M + r + B).$$

During a traced run the oracle can temporarily retain proofs of discovery queries that will not be emitted; this uses the analogous bound with $Q + Q_c$. The implementation does not promise a minimal-memory streaming certificate producer. That is an optimization opportunity, not an unaccounted exponential allocation.

6.2 Why the sparse support is polynomial for interval inputs

It would be incomplete to leave the geometry-independent parameter s entirely unexplained. For arbitrary positive Boolean functions it may be $2^r - 1$. For the present interval-encoded inputs, classical weighted orbit theory gives a stronger conclusion.

Lemma 6.2 (Polynomial signature support from weighted AHT). *For k interval pairings and r interval-union ports with a total of M intervals, the number s_0 of positive component signatures is bounded by a polynomial in $k + M + r + B$.*

Proof. The case $N = 0$ is immediate. For $N > 0$ and $r > 0$, assign to each point the nonnegative vector

$$w(x) = (\mathbf{1}_{A_1}(x), \dots, \mathbf{1}_{A_r}(x)) \in \mathbb{Z}_{\geq 0}^r.$$

The vector is constant on a common partition with at most $2M + 1$ nonempty cells, since changes can occur only at port endpoints. The partition and its vectors have a polynomial-size representation, for example by an endpoint sweep keeping an r -bit active-port mask. For a component O , its orbit-weight vector is

$$v(O) = \sum_{x \in O} w(x).$$

Its positive coordinates are exactly $\sigma(O)$.

Apply the weighted AHT theorem to this piecewise constant vector weight. Its compressed output is a list of interval blocks, each block representing many components sharing one orbit-weight vector [1, Section 6]. The number of written blocks is polynomial in the input parameters. The weight dimension is r , the number of input cells is at most $2M + 1$, and a safe numerical weight bound is $D = 1 + rN$, so $\log D = O(B + \log(r + 1))$. A signature can occur only as the positive-coordinate support of one of the vectors on those output blocks. The number of distinct such supports is at most the number of blocks, and hence is polynomial. When $r = 0$, there is at most the empty signature and the conclusion is immediate. $\square$

The proof uses the *compressed output format*, not an impossible explicit list of all C components. A block may stand for 2^{16000} components. One must also use a coordinate per supplied port, not one coordinate per point. The theorem's polynomial dependence on weight dimension is precisely what permits polynomially many ports.

Theorem 6.3 (Polynomial sparse interval-incidence observer). *Given binary-encoded interval pairings and interval-union ports, the complete sparse signature histogram can be computed and furnished with independently replayable certificates in polynomial bit time in the encoded input size. There is no requirement that $r = O(\log n)$, $r = O((\log n)^2)$, or that component multiplicities be polynomially bounded as ordinary numbers.*

Proof. Lemma 6.2 gives polynomial s_0 . The number of sparse requests in Theorem 4.3 is at most $O(1 + sr)$; the certificate request bound is of the same polynomial order. Classical unweighted AHT gives polynomial P, V, L for the coned instances. Substitute these bounds into Theorem 6.1 and the certificate-storage estimate. $\square$

This theorem is a consequence of established weighted AHT combined with the explicit unweighted-query reduction. It should not be advertised as a new general complexity theorem for weighted orbit extraction. Its practical advantage is that the executable path uses the already available unweighted proof engine. A direct weighted implementation may have better constants and would additionally return coordinate magnitudes, not just their supports.

6.3 Dense output remains an exponential requirement

If a caller insists on an array with an addressable slot for every subset, writing that array costs $\Omega(2^r)$ slots regardless of the preceding theorem. Even a zero universe with r empty ports has a dense histogram of that length. Thus the theorem changes the output interface rather than claiming to fill an exponentially long object in polynomial time.

The distinction also explains the dense benchmark control. Giving every one of the 2^r signatures a static point is possible, but the corresponding bit-pattern interval marks have exponentially many interval pieces. Such a family is not a counterexample to polynomiality in $k + M + r + B$.

7 Incidence refined by a binary orientation character

7.1 Order reversal and orientation parity are different inputs

Attach to each generating pairing an additional bit $p \in \{0, 1\}$. This bit records the supplied character, not necessarily the sign of the interval isometry. The double cover has points (x, e) , $e \in \{0, 1\}$, with relations

$$(x, e) \sim (g(x), e + p).$$

It is encoded in two consecutive blocks of N points, using $2k$ interval pairings. Each port is lifted to both sheets, so the number of marked interval pieces is at most $2M$. This encoding doubles the size numerically but adds only one endpoint bit.

A base component is *consistent* if every closed walk in its relation graph has total parity zero. If the supplied bits are the actual surface orientation character, consistency means orientability. Without that provenance, only the binary-label consistency statement is justified.

Lemma 7.1 (Lift support is unchanged). *Every connected component of the double cover projects surjectively onto one base component and has its exact port signature. A consistent base component has two lifted components; an inconsistent one has one. Consequently, if $b(T)$ and $\ell(T)$ are the base and lifted histograms, respectively, then*

$$\text{supp } b = \text{supp } \ell, \quad b(T) \leq \ell(T) \leq 2b(T).$$

Proof. Lift a path from a chosen base point to any other point in its component. Starting at a chosen sheet determines a lifted endpoint, proving surjectivity of each lifted component. Because a port is lifted on both sheets, a lifted component meets that port exactly when its base component does.

If all closed walks have parity zero, assigning a sheet label by path parity is path-independent. The two possible initial labels give two disjoint components. If a closed walk has parity one, its lift joins the two sheets above one point, and path lifting then joins them above every point. The cover is connected over that base component. Summing by signature proves the bounds. $\square$

Theorem 7.2 (Typed sparse incidence and support reuse). *Let $o(T)$ and $u(T)$ count consistent and inconsistent base components of signature T . Then*

$$o(T) = \ell(T) - b(T), \quad u(T) = 2b(T) - \ell(T).$$

After the base sparse support has been recovered, the lifted histogram can be recovered with one baseline and at most s_0 additional zeta requests, before caching. No second balanced support search is necessary. Typed incidence has polynomial bit complexity for interval-union marks.

Proof. Lemma 7.1 gives $b = o + u$ and $\ell = 2o + u$, whose solution is the stated formula. Order the known support by increasing cardinality. For a support T in that order, every positive proper-subset support is already known and processed. Since the lifted support equals the base support, its weight is

$$\ell(T) = G_{\text{lift}}(T) - \sum_{S \subsetneq T: b(S) > 0} \ell(S).$$

This triangular calculation uses one value per known support. The total lifted count supplies a final consistency check. For independent certification, the ordinary sparse certificate criterion is applied to both histograms; it may require additional deletion values, all under the shared budget. Polynomiality follows from Theorem 6.3 on the doubled encoding. $\square$

The implementation’s signed result contains rows $(T, o(T), u(T))$, preserving zeros in the last two fields when the other field is positive. Its proof contains separately replayable base and cover certificates. The independent signed verifier reconstructs the cover from the supplied signed pairings, checks both certificates, checks equality of supports, and recomputes the two nonnegative differences. A genuine character-changing input mutation cannot be hidden by retaining the old cover proof.

8 Counterexamples to overstrong conclusions

8.1 Signed weights invalidate zero-based extraction

For the abstract signed histogram

$$h(\{1\}) = 1, \quad h(\{2\}) = -1,$$

the total zeta value at $\{1, 2\}$ is zero although the histogram is not zero. A total-mass stopping rule would fail. This is why the reconstruction theorem is about *nonnegative component counts*; it must not be transferred unchanged to Euler-characteristic coefficients, signed polynomial coefficients, or arbitrary integer-weight cancellation data. In Section 7, subtraction is performed only after two genuine nonnegative histograms have been recovered and verified.

8.2 Equal incidence is not equal future attachment behavior

Proposition 8.1 (Four-point attachment obstruction). *There exist two interval relations with identical complete port-incidence histograms that acquire different component counts after the same new pointwise attachments.*

Proof. Take four points a_1, a_2, b_1, b_2 and ports $A = \{a_1, a_2\}$ and $B = \{b_1, b_2\}$. In the first relation E , identify $a_1 \sim b_1$ and $a_2 \sim b_2$. In the second relation E' , identify $a_1 \sim b_2$ and $a_2 \sim b_1$. Both histograms consist of the single row $h(\{A, B\}) = 2$. Add to each relation the same two attachments $a_1 \sim b_1$ and $a_2 \sim b_2$. The first relation still has two components; the second becomes connected. These are valid interval inputs by assigning the four consecutive coordinates 0, 1, 2, 3 and using singleton pairings. $\square$

Thus a signature histogram is an exact observer, not a complete state for arbitrary future gluing. It loses the association between individually named points, as well as multiplicity at a port, cyclic order, slope, and boundary attachment maps. A hierarchy algorithm may use the histogram to answer the specific incidence question, but cannot replace the original boundary state by that histogram without a separate congruence theorem for its allowed attachments.

8.3 Polynomial local work does not bound hierarchy search

A complete recognition bound also needs an algorithm for producing the supplied interval relations from the input diagram, controlling the encoding of cut surfaces and marked boundaries, generating the necessary candidates, and stopping after a bounded number of stages. None of those facts follows from an incidence query theorem.

Lackenby’s 2026 hierarchy preprint supplies important geometric algorithms, but its Section 9 explicitly distinguishes counts of steps from fully specified running-time estimates and leaves

parameters to be bounded there [3]. Treating that discussion as an already available complete quasi-polynomial implementation would skip precisely the cost obligations that the repository is trying to resolve.

9 A precise interface to a future quasi-polynomial hierarchy

Proposition 9.1 (Composition accounting). *Suppose a proposed hierarchy procedure on an n -crossing input makes at most $H(n)$ sparse-incidence calls. Suppose every supplied incidence instance has encoded length at most $L(n)$, and that all other work, including candidate production, port extraction, certificate replay, and terminal tests, takes at most $W(n)$ bit operations. Then its total work is bounded by*

$$W(n) + H(n)L(n)^{O(1)}.$$

If W, H, L are each at most $2^{(\log(n+2))^{O(1)}}$, the resulting bound is quasi-polynomial. This conclusion assumes correctness and completeness of the underlying hierarchy procedure; it does not supply them.

Proof. Theorem 6.3 gives a fixed polynomial bound for each incidence call, including its chosen proof operations. Sum over the $H(n)$ calls and add the remaining work. Products and fixed powers of the displayed quasi-polynomial scales remain in that scale. $\square$

The point is not the elementary summation. It is that the incidence observer no longer contributes an independent $2^{r(n)}$ factor. Polynomially many ports are permissible when their interval encodings are polynomially large. For the project’s more specific target $n^{O(\log n)} = 2^{O((\log n)^2)}$, one needs compatible $O((\log n)^2)$ exponents for the *whole* expression, not merely membership in the broader quasi-polynomial class.

Depth and number of calls are also different quantities. A search tree of depth $O(\log n)$ and branching factor $n^{O(1)}$ has quasi-polynomially many nodes, not necessarily polynomially many. If branching or encoded surface sizes grow faster, a logarithmic depth slogan does not repair the bound. The interval-incidence theorem removes one obstacle; it does not replace this global accounting.

10 Implementation contracts and resource behavior

10.1 Module separation

The source package has a small mathematical core. `reconstruct.py` implements flat and balanced positive-zeta extraction independently of interval geometry. `geometry.py` validates and normalizes public input. `oracle.py` performs canonical union queries through the unchanged AHT kernel. `verify.py` checks sparse certificates without importing discovery. `signed.py` implements the binary cover, support reuse, and typed replay. `api.py` provides the ordinary sparse interface and the dense ablation.

The kernel loader checks the exact Git blob hash of each vendored source file before importing it. The producer and verifier are loaded separately. This protects the claimed source identity and makes an accidental edited fixture visible; it is not a claim that source hashing proves the algorithm correct.

10.2 Completion, interruption, and mutable state

A successful query returns `status='COMPLETE'`, its full sparse histogram, and an optional certificate. A query interrupted by its support, total-query, or total-AHT-cycle allowance returns `INCONCLUSIVE` with no histogram, no total-count assertion, and no certificate. It may retain work statistics. There is no conversion from resource exhaustion to “unknot” or “knotted.”

The total cycle budget is shared among the baseline, all coned calls, and additional certification calls. The signed wrapper shares it between the base and the doubled system as well. The same is true for the total query allowance. AHT cycles are structural events, not a wall-clock or bit-operation meter. The optional callback provides cooperative cancellation, and its exceptions propagate rather than being mislabeled as a successful mathematical result.

All core objects are constructed from normalized copies. The supplied input lists are not mutated. A complete sparse answer produced before a failed certificate stage is not exposed as a fully certified answer by the combined API; the combined operation returns inconclusive instead. A caller needing separate uncertified and certified products can use separate explicit calls with separate documented budgets, rather than silently resetting one budget.

10.3 Serialization and hostile-input scope

Endpoint integers and component counts may exceed Python’s default limit for decimal-string conversion. The transport layer uses exact hexadecimal strings for sufficiently large integers, without modifying the interpreter’s global limit. Boolean values are not accepted as mathematical integers. The proof format carries no floating-point quantity used in a correctness decision.

Polynomial verification in the explicit proof length does not mean that any untrusted JSON file is cheap to parse or store. Callers should impose limits on serialized bytes, nesting, and wall time before accepting external certificates. The delivered verifier is a mathematical checker, not a sandbox for arbitrarily large hostile inputs.

11 Exact validation and controlled measurements

11.1 Completed validation

The following checks were completed in this environment. The standard-library suite contains 33 test methods, including invalid input, resource-boundary, proof-mutation, and noninterference tests. Its exhaustive subtest checks all $3^8 = 6561$ histograms with eight possible supports on three ports and weights in $\{0, 1, 2\}$, using both flat and balanced extraction. Three hundred additional positive histograms exercise larger masks and multiplicities. Tests also check the explicit balanced-tree bound and the logarithmic-query behavior of a single hidden singleton on 1024 ports.

A separate seeded audit generates 1000 interval systems with up to 50 points and up to ten ports. It computes components independently by literal union-find and compares every sparse entry. It also assigns random binary parities and checks the typed result against an independent graph traversal with parity labels. Every supplied base and cover certificate is replayed. The audit retains all inputs, expected sparse outputs, and statistics in `results/validation.json`. All 1000 ordinary comparisons and all 1000 binary-character comparisons agree, with no failed proof replays.

The ordinary certificates in that audit exercise every supported local AHT rule: deletion, contraction,

trimming, merger, transmission, and truncation. The exact event counts are in the audit file. This is finite evidence about the implementation, not a replacement for the general proofs. The test log also includes a 2^{16000} -point transport/replay example and a large parallel chain family without point expansion.

Adversarial tests alter multiplicities, omit supports, duplicate entries, change port bindings, truncate local traces, delete or duplicate union proofs, insert an unused otherwise valid proof, and replace an integer with a Boolean. They check a specific case in which certificate construction needs one more orbit query than discovery: the shared allowance stops it rather than resetting. The four-point attachment obstruction is an executable regression.

11.2 Benchmark design

Nine supplied interval workloads are measured with five arms: cached dense inversion, an identical dense A/A control, flat sparse recovery, balanced sparse recovery, and balanced recovery with certificate production and independent replay included in its timing. Each arm starts with fresh oracle state. There is one warmup and seven measured rounds per workload, and arm order is shuffled with seed 261008491. Thus the principal experiment contains 45 warmup and 315 measured calls. Result comparison is outside the timed region; replay is inside the timed region only for the explicitly labeled proof/replay arm.

The implementation uses CPython; its full version and platform string are recorded in `results/benchmark.json`, along with source hashes, every sample, arm order, query count, and proof size. Timings are observations from this container, not machine-independent complexity claims. The table reports median milliseconds and the median of the paired dense/balanced ratios. A paired ratio need not equal the ratio of the two separately displayed medians.

Workload	Dense	Flat	Balanced	Proof/replay	Ratio
Static, coincident	0.683	0.054	0.073	0.180	9.25
Static, nested	0.763	0.300	0.369	0.884	2.05
Static, disjoint	20.130	1.579	0.783	1.638	25.44
8-block chain, coincident	0.560	0.450	0.476	0.682	1.21
8-block chain, nested	1.879	1.837	1.799	2.456	1.01
8-block chain, disjoint	22.849	6.541	4.039	5.389	5.70
16-block chain, disjoint	380.408	47.020	25.777	28.915	14.70
All 32 signatures	3.134	3.477	3.877	6.824	0.86
Small mixed relation	0.294	0.244	0.267	0.589	1.10

Table 1: Completed same-kernel component measurements. Times are milliseconds. “Ratio” is paired dense time divided by balanced sparse time; values below one indicate a slowdown. No row is a whole-knot recognition benchmark.

The static workloads have universe size 2^{4000} and eight ports. The chain workloads link consecutive equal blocks by translations. Their base orbits are the common offsets within a block, so a block length of 2^{500} represents that many parallel components. The eight-block chains use six ports; the sixteen-block chain uses eight ports and block length 2^{128} . Marks are explicit coordinate intervals with coincident, nested, or disjoint layouts. They are not asserted to have been extracted from a native knot-exterior attachment construction.

Workload	r	s_0	Dense calls	Sparse calls	Proof KiB
Static, coincident	8	2	2	2	26.4
Static, nested	8	8	9	9	89.2
Static, disjoint	8	8	256	21	179.6
8-block chain, coincident	6	2	2	2	18.4
8-block chain, nested	6	6	7	7	52.1
8-block chain, disjoint	6	6	64	14	66.2
16-block chain, disjoint	8	8	256	21	48.8
All 32 signatures	5	32	32	32	19.0
Small mixed relation	3	4	8	8	3.3

Table 2: Actual orbit calls, including the baseline. Here s_0 includes a positive empty signature. Proof size is the JSON byte count divided by 1024; it is a transport measurement, not a claim of minimal binary encoding.

11.3 Interpretation, including regressions

The disjoint sparse cases are the clearest demonstration of the mechanism. Dense inversion needs every distinct subset union, whereas balanced discovery asks a small selection of them. The eight-port disjoint examples reduce 256 count calls to 21. The six-port disjoint chain reduces 64 to 14. These are exact call counts, independent of timing noise.

Coincident and nested marks illustrate a different effect. Canonical-union caching already reduces the number of underlying count calls, so sparse recovery cannot improve those counts much or at all. It instead avoids dense mask enumeration and inversion. On the eight-block nested chain, the result is near a tie; certificate production and replay add visible cost. The all-signatures control has 32 positive rows and 32 count calls in either representation. Sparse bookkeeping adds overhead there. The small mixed case is also too small and noisy to justify a broad speed claim.

The A/A samples are retained specifically to expose this scale of variation. No aggregate claim that the complete recognizer has accelerated is inferred from these tables. The observer is not called by ordinary knot dispatch in the inspected pipeline, and the dense baseline is a controlled ablation rather than a rerun of that pipeline.

An exploratory classical-kernel benchmark containing a 32-block chain was stopped by this session’s execution-time limit before a complete result file was produced. It contributes no numbers to the tables. The delivered benchmark uses the explicitly named 16-block case and completed all its rounds. This change concerns experimental tractability, not the theorem or the compressed input model.

11.4 Large-port scaling without a dense allocation

For disjoint static ports, the universe has size 2^{500} and the ports partition it into equal blocks for the listed powers of two. The exact histogram has one singleton signature per port. For the one-active-port family, the universe has size 2^{16000} , exactly one port is the entire universe, and the remaining ports are empty. The unique nonempty signature has multiplicity 2^{16000} . In the 1024-port case, balanced discovery uses 21 block tests and only two distinct underlying orbit counts after union reuse. No dense array with 2^{1024} entries was constructed or timed.

Family	r	Dense slots	Sparse rows	Calls	Time (ms)
Disjoint static	16	2^{16}	16	49	1.610
Disjoint static	32	2^{32}	32	113	4.867
Disjoint static	64	2^{64}	64	257	15.431
Disjoint static	128	2^{128}	128	577	53.372
One active port	64	2^{64}	1	2	0.118
One active port	256	2^{256}	1	2	0.309
One active port	1024	2^{1024}	1	2	1.404

Table 3: Completed balanced-only scaling runs, seven repetitions each. The “dense slots” column is the output-format requirement, not a measured dense run or an extrapolated timing.

12 Integration plan and preservation of existing guarantees

The package is designed as an additive research directory, for example `Topology/UnknotRecognition/research/sparse_incidence_20261008/`. It does not overwrite any maintained source and does not allocate a numbered report slot that might already have been used in a concurrent repository update. `INTEGRATION.md` gives the precise production-facing steps.

A safe first integration is an explicit sparse observer API, separate from ordinary knot recognition. It should accept the same interval-pairing model, normalize ports once, and return a tagged sparse histogram. Existing callers expecting dense mask indexing must either remain on the dense interface or request a deliberate, separately capped expansion. Silent shape changes are not compatible with exact callers.

The certificate schema should remain distinct from the existing dense incidence schema. A dense verifier’s 2^r proof-index contract is not an appropriate compatibility layer for the new sparse result. The proof layer can reuse the maintained local orbit verifier after its exact version and encoding contract are checked. The included source-probe script reports blob matches and fails on mismatches rather than claiming to have validated an unseen future revision.

Before promotion, run the full maintained tests and the native normal-arc adapter audits on the intended checkout, including geometric provenance tests. Measure sparse and dense observers on the same native inputs, retain A/A controls, and keep whole-recognition timings separate. Until such a production caller is introduced, the correct performance statement is a faster supplied incidence query, not a faster complete recognizer.

13 Further research questions and concrete next targets

13.1 A weighted engine versus repeated unweighted queries

The polynomial support theorem comes from weighted AHT, while the executable method makes repeated unweighted calls. A directly certified vector-weight transfer engine could compute all port counts, component Euler data, and incidence in one pass. The concrete target is an independently replayable weighted-transfer contract with controlled integer growth and a paired comparison against the present reduction. The relevant question is not whether a polynomial algorithm exists—that is classical—but whether one can obtain a smaller practical proof and lower total cost within the maintained architecture.

13.2 Sharper support bounds tied to geometry

The weighted theorem supplies a polynomial upper bound without a small explicit exponent in this paper. Can normal-arc systems or actual hierarchy ports force a much smaller bound, such as a near-linear number of signature types in the number of pairing and marking intervals? A useful approach is to count how many different supports can be created by weighted transfer and static contraction, then test extremal examples generated by genuine surface encodings. No such near-linear claim is assumed here.

13.3 Residual-query data structures

The delivered code scans every previously recovered support when evaluating a residual. This gives a conservative $Q(s+1)(r+B)$ term. A dynamic sparse subset-sum structure, compressed trie, or rank-separated index could reduce that term on structured supports. The target must remain exact and should be benchmarked against the list scan, which is inexpensive for very small s . An index whose preprocessing enumerates the Boolean cube would defeat the purpose.

13.4 Cost-aware balanced trees and port equivalences

Balanced deletion minimizes neither total coned interval count nor total AHT cycles. Can one choose a binary partition tree using port overlap, repeated unions, or measured query cost, while retaining a provable approximation bound for an explicit cost model? Identical ports may be quotiented before search, and their bits restored afterward. Any more aggressive equivalence must prove equality of component-incidence behavior rather than merely similarity of coordinate ranges.

13.5 Certificate size and streaming replay

Discovery and independent verification ask overlapping but not identical sets of questions. Can a streaming producer retain just the proofs that will be needed by the final sparse certificate without a costly restart? Can a certificate verifier arrange the sparse equations to reduce repeated subset arithmetic, while preserving the immediate-deletion information lower bound? The implementation already rejects unused proof records, but it does not minimize temporary traced storage.

13.6 Finite covers beyond a binary character

For a fixed finite group of order q , a regular cover similarly preserves port signatures when every mark is lifted completely. A component lifts to between one and q components. The support-known triangular reconstruction still applies. Determining detailed monodromy subgroup types would require additional data or multiple covers; the two-by-two linear formulas of Section 7 do not automatically generalize to identify them. A precise next target is a small fixed-group family with a proved separating set of covers and independently replayable subgroup-type counts.

13.7 Attachment-compatible refinements

Proposition 8.1 shows that incidence alone is not a congruence for pointwise attachments. What additional compressed data make it a congruence for the *particular* attachments occurring in a normal-surface hierarchy? Candidate data include boundary multiplicities, ordered interval maps,

permutation actions on repeated sheets, and a finite set of slope labels. The goal is a closure theorem: two summaries declared equivalent must remain equivalent under every allowed continuation. This is more important than showing that an isolated histogram is small.

13.8 Certified port extraction and compressed cutting

The local polynomial theorem requires ports given by polynomially many binary intervals. A future geometric adapter must prove that this form is preserved by cutting, crushing, and provenance updates. A successful adapter should supply a certificate connecting every new marked interval to the original triangulation and to the chosen normal coordinates, including orientation characters. A compact normal-coordinate vector alone does not prove that an arbitrary proposed collection of attachment ports is correct.

13.9 Global potential and search accounting

The decisive missing result is a potential controlling the number of hierarchy states and the size of their complete encodings. A meaningful theorem would bound $H(n)$ and $L(n)$ in Proposition 9.1, and account for candidate generation and terminal checks in $W(n)$. Geometric depth, branching, compressed word length, and numerical surface weight should have separate counters. The current contribution makes one of these costs polynomial; it does not allow the remaining counters to be omitted.

13.10 Native validation and formal proof boundaries

The next experimental step is a native normal-surface corpus with adversarial port arrangements, compared against both the maintained dense observer and an independent geometric component extractor at feasible sizes. Whole-knot measurements should begin only when a production caller actually uses the new query. A tractable formalization target is the positive residual invariant and Theorem 5.1 over a finite Boolean lattice, separated from AHT and geometric provenance. That would make the sparse completeness argument machine-checkable without pretending to formalize an entire hierarchy at once.

14 Conclusion

The maintained dense marked-incidence observer admits an exact sparse replacement that reuses certified unweighted interval-orbit counting. Positive residuals locate minimal unknown signatures, whole multiplicities are recovered at once, balanced deletion makes discovery sensitive to support size, and local deletion equations plus total mass independently certify the complete result. A binary cover preserves support and permits a second histogram to be recovered without repeating support discovery.

The asymptotic conclusion for supplied interval data is polynomial, not merely conditional on a logarithmic port bound. Its geometric support argument is an explicit corollary of classical weighted AHT, duly attributed. The observed speedups concern exactly this observer and are accompanied by dense and small controls showing overhead. For general unknot recognition, the remaining research burden is a certified attachment-compatible hierarchy with controlled candidate generation, encoded

sizes, and total work. The delivered primitive removes a concrete exponential-format cost from that program without concealing those unresolved obligations.

A Executable usage and artifact inventory

From the extracted package root:

```
export PYTHONPATH="$PWD/src${PYTHONPATH:+:$PYTHONPATH}"
python -m unittest discover -s tests -v
python experiments/validate.py
python experiments/benchmark.py --rounds 7
python experiments/make_tables.py
python -m sparse_incidence analyze examples/mixed.json \
    --output results/mixed.json --certificate
python -m sparse_incidence verify examples/mixed.json results/mixed.json
cd paper && sh build.sh
```

No network connection is required. Python 3.10 or later is the intended minimum; the actual tested version is recorded in the results. Rebuilding the article requires a LaTeX installation providing the packages in its preamble and BibTeX. The example commands analyze an interval relation, not a knot diagram.

Path	Contents
paper/article.tex, article.pdf	Complete article and compiled PDF.
paper/references.bib	Primary literature and immutable repository references.
src/sparse_incidence/	Sparse extraction, geometry, cached oracle, independent replay, signed cover, CLI.
vendor/	Two unchanged upstream modules, separately pinned by Git blob and SHA-256.
tests/test_sparse.py	33 unit-test methods with exhaustive, seeded, and adversarial subcases.
experiments/	Literal graph controls, seeded audit, fresh-state paired benchmark, table generation.
results/	Completed test log, all audit inputs/results, raw timing samples, summary logs.
examples/	Ordinary and huge-multiplicity inputs and independently replayed certificates.
integration/	Local source-identity probe; no remote writes or automatic integration.
CLAIMS.md, INTEGRATION.md	Scope ledger and production-review obligations.
PROVENANCE.json	Repository ref, inspected sources, source hashes, runtime and scope.
SHA256SUMS	Delivered-file integrity manifest, excluding itself.

B Worked sparse certificate on a mixed relation

The small mixed example has $N = 10$, the translation $[0, 4] \rightarrow [5, 9]$, and ports

$$A_1 = [0, 2), \quad A_2 = [6, 9), \quad A_3 = [0, 1) \cup [9, 10).$$

The five components are $\{i, i + 5\}$, $0 \leq i < 5$. Their signatures give

$$h(\{2\}) = 2, \quad h(\{3\}) = 1, \quad h(\{1, 2\}) = 1, \quad h(\{1, 3\}) = 1,$$

and all other entries vanish. In particular $h(\emptyset) = 0$. A cardinality-ordered certificate first proves the two singleton entries. It then checks the weight at $\{1, 2\}$ after subtracting the known singleton mass 2, and checks both deletion values. The entry at $\{1, 3\}$ is handled similarly after subtracting the singleton mass 1. The final total $2 + 1 + 1 + 1 = 5$ rules out every missing signature, including $\{2, 3\}$ and $\{1, 2, 3\}$, without a dense forward transform.

The standalone proof file contains full local AHT witnesses for its actual coned unions. A reader can mutate the claimed row weights or bindings and replay the file using the CLI. The sparse verifier does not trust the **COMPLETE** label in a surrounding result; it verifies the certificate against the separately supplied input.

C Claim ledger

Claim	Status
Coned-union identity	Proved; existing maintained construction, reused and independently reconstructed in replay.
Positive sparse extraction	Proved for every nonnegative Boolean histogram; implemented in two strategies.
Support-sensitive balanced bound	Proved with an explicit binary-tree count; checked on exhaustive and seeded examples.
Sparse local completeness certificate	Proved; independently implemented and adversarially tested.
Polynomial interval-incidence complexity	Proved as a consequence of classical weighted and unweighted AHT plus the explicit sparse reduction. Not a new general weighted-orbit theorem.
Binary-character refinement	Proved and implemented, including known-support cover recovery and shared budgets.
Polynomial general unknot recognition	Not claimed.
General quasi-polynomial unknot recognition	Not established by this contribution.
Whole-knot practical acceleration	Not measured; no production recognition dispatch was changed.
Native surface-adaptor validation	Not run here; explicitly required before production promotion.
Full upstream suite	Not run here. The 33-method delivered suite and 1000-case independent audits were run.

Claim	Status
Formal machine-checked proofs	Not provided; written proofs and executable exact checks only.
Priority beyond the inspected repository	Not claimed; related weighted-orbit and hidden-hypergraph literature is acknowledged.

References

- [1] Ian Agol, Joel Hass, and William Thurston. The computational complexity of knot genus and spanning area. *Transactions of the American Mathematical Society*, 358(9):3821–3850, 2006. doi: 10.1090/S0002-9947-05-03919-X. URL <https://arxiv.org/abs/math/0205057>. Especially Theorems 12 and 16 and the compressed weighted-output format in Section 6.
- [2] Nader H. Bshouty and Hanna Mazzawi. Optimal query complexity for reconstructing hypergraphs. In *27th International Symposium on Theoretical Aspects of Computer Science*, volume 5 of *Leibniz International Proceedings in Informatics*, pages 143–154. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2010. doi: 10.4230/LIPIcs.STACS.2010.2496. URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.STACS.2010.2496>.
- [3] Marc Lackenby. Incompressible surfaces, hierarchies and unknot recognition, 2026. URL <https://arxiv.org/abs/2607.23350>. Version 1, 25 July 2026. Section 9 explicitly separates step counts from fully specified running times.
- [4] ProveIt contributors. Independent orbit proofs and reusable marked-union queries, 2026. URL https://github.com/VladimirReshetnikov/ProveIt/blob/7518823550fbc8c217bc8be0113fe52002e77465/Topology/UnknotRecognition/synthesis/orbit_certificates.tex. Maintained `synthesis/orbit_certificates.tex`, Git blob `a83b3ab44d95b2e8c35752266d9db626ca126225`.
- [5] ProveIt contributors. ProveIt: Topology/UnknotRecognition, 2026. URL <https://github.com/VladimirReshetnikov/ProveIt/tree/7518823550fbc8c217bc8be0113fe52002e77465/Topology/UnknotRecognition>. Inspected repository ref `7518823550fbc8c217bc8be0113fe52002e77465`. Exact kernel blob identities are recorded in the artifact provenance.